REST API CLIENT USING JAVA

1. Abstract

REST APIs are widely used for communication between applications and web services. This project demonstrates how a Java application can interact with a REST API, send HTTP requests, receive JSON responses, and display the retrieved information. The project uses Java 17's HttpClient API to establish communication with a public REST API. The application fetches user data from an online service and displays the response in the console. This project helps in understanding web services, APIs, HTTP protocols, and JSON data exchange.

---

2. Objective

The objectives of this project are:

- To understand the concept of REST APIs.
- To learn how to send HTTP requests using Java.
- To receive and process JSON responses.
- To understand client-server communication.
- To gain practical experience with web services.

---

3. Software Requirements

Hardware Requirements

- Processor: Intel i3 or above
- RAM: 4 GB or above
- Storage: 500 MB free space

Software Requirements

- Operating System: Windows/Linux/MacOS
- Java Development Kit (JDK 17)
- Command Prompt (CMD)
- Notepad or any text editor
- Internet Connection

---

4. Introduction

Modern applications frequently communicate with web services to exchange information. REST (Representational State Transfer) APIs provide a standard way for applications to interact over the internet using HTTP methods such as GET, POST, PUT, and DELETE.

In this project, a Java application acts as a client and sends an HTTP GET request to a public API. The API responds with JSON data, which is displayed to the user. This demonstrates how Java applications can consume external web services.

---

5. Concepts Used

REST API

A REST API is a web service that allows applications to communicate using HTTP requests.

HTTP GET Request

The GET method is used to retrieve data from a server.

HttpClient

HttpClient is a Java API used for sending HTTP requests and receiving responses.

JSON

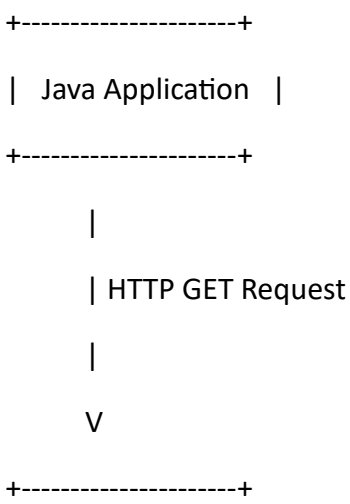
JSON (JavaScript Object Notation) is a lightweight format used for data exchange.

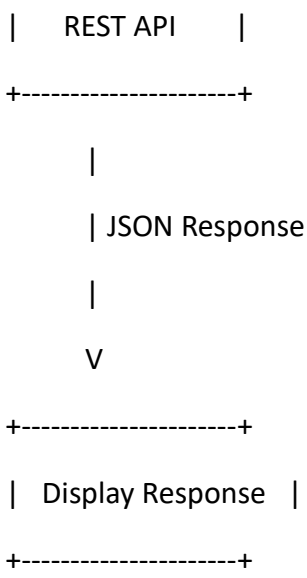
Exception Handling

Used to handle runtime errors and network-related exceptions.

---

6. System Architecture





The Java application sends a request to the REST API and receives JSON data as a response.

---

7. Algorithm

1. Create an HttpClient object.
2. Create an HTTP GET request using the API URL.
3. Send the request to the API server.
4. Receive the response from the server.
5. Extract the status code.
6. Display the JSON response.
7. Handle exceptions if any errors occur.

---

8. Source Code

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class RestApiClient {
```

```
    public static void main(String[] args) {

        try {

            HttpClient client = HttpClient.newHttpClient();

            HttpRequest request = HttpRequest.newBuilder()
                    .uri(URI.create("https://jsonplaceholder.typicode.com/users/1"))
                    .GET()
                    .build();

            HttpResponse<String> response =
                    client.send(request, HttpResponse.BodyHandlers.ofString());

            System.out.println("Response Status Code: "
                    + response.statusCode());

            System.out.println("\nJSON Response:");
            System.out.println(response.body());

        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

---

9. Sample Output

Response Status Code: 200

JSON Response:

{

"id": 1,

"name": "Leanne Graham",

"username": "Bret",

"email": "Sincere@april.biz"

}

(Attach the screenshot of your CMD output here.)

---

10. Applications

- Weather Forecast Applications
- Online Shopping Platforms
- Banking Applications
- Social Media Platforms
- Travel and Booking Systems
- Mobile Applications
- Cloud Services

---

11. Advantages

- Fast communication with web services.
- Easy integration with external systems.
- Real-time data retrieval.
- Platform independent.
- Supports scalable applications.

---

12. Limitations

- Requires internet connectivity.
- Depends on API availability.
- Public APIs may have usage limits.
- Security measures are not implemented in this basic project.

---

13. Future Enhancements

- Parse JSON and display formatted output.
- Connect to authenticated APIs.
- Add GUI using Java Swing or JavaFX.
- Support POST, PUT, and DELETE requests.
- Store API responses in a database.

---

14. Conclusion

The REST API Client project successfully demonstrates how Java applications communicate with web services using HTTP requests. The application retrieves JSON data from a public API and displays the response. This project provides practical exposure to REST APIs, HTTP communication, and modern Java networking features.

---

15. Viva Questions and Answers

Q1. What is a REST API?

A REST API is a web service that allows communication between applications using HTTP methods.

Q2. What is JSON?

JSON (JavaScript Object Notation) is a lightweight data format used for exchanging information between systems.

Q3. What is HttpClient?

HttpClient is a Java API used to send HTTP requests and receive HTTP responses.

Q4. What is an HTTP GET request?

GET is an HTTP method used to retrieve data from a server.

Q5. What does status code 200 indicate?

It indicates that the request was successful.

Q6. What package contains HttpClient?

The java.net.http package.

Q7. What is the purpose of URI?

URI identifies the location of a resource on the internet.

Q8. Why are APIs important?

APIs allow different applications and systems to communicate and exchange data.

Q9. What happens if the API server is unavailable?

The application may throw an exception or fail to receive a response.

Q10. What are the advantages of REST APIs?

REST APIs are lightweight, scalable, easy to use, and widely supported.